M6 — Scaling Writes by Splitting the Data

Campus Pulse on two hash-partitioned Supabase databases

Team: Fupei Guo, Yanyu Zhou, Junru Zhou
CSCE 553 • github.com/GFP-1011/campus-pulse-csce553 (branch `m6`)

Measurements: 10 November 2026 (UTC)

Summary

- **Write bottleneck.** We extended Campus Pulse so that one "publish" writes 46 rows: the post, its hashtags, a real-time full-text (GIN) index entry, and home-feed copies for the author and 40 followers. On one Supabase Nano database, publish throughput peaked at **574 posts/s** (25.6k rows/s) at 64 clients and then *declined* to about 500 posts/s, with p99 up to 2.9 s. At the peak the database had all 15 pooled server connections busy, mean `publish_post()` time had risen 5× (3.4 to 16.8 ms), 3–4 backends were waiting on WAL write/flush, and the VM was about 80% busy+iowait. The same API served 1,571 feed reads/s, so the API was not the limit.
- **Shard key and models.** We chose the owning user's id (UUID) hashed into 64 buckets (Section 1). The integer M1 schema became a UUID schema in which every child row carries its owner, with follow edges on both sides, feed copies on the reader's shard, and a directory (email → id) and a bucket map on shard_0 (Section 2).
- **Ten steps** implemented on two us-west-2 Supabase projects with the adapted handout SQL (Section 3). Checks: 181/219 users per shard, 0 rows on the wrong shard, the wrong-bucket insert was rejected, and 16/16 end-to-end API checks of the Step 9–10 write and read paths pass.
- **After sharding.** Peak **744 posts/s** (1.30×), with a median of 700 vs. 501 posts/s over the saturated stages (1.40×). At equal throughput each shard carries about half the rows and runs `publish_post()` about 5× faster. The gain is below 2× because cross-shard feed copies keep one commit per post on *each* shard, and because the second round trip doubles latency at low load (p50 78 → 153 ms). Likes, which touch one owner, gained 21% and lost their commit waits.

1 Sharding Worksheet: Choose Your Shard Key

Team: Fupei Guo, Yanyu Zhou, Junru Zhou App: ☐ E-commerce ☒ Social media ☐ Event management
Date: 10 November 2026

The busy-hour rates below use the M2 workload model scaled to the 1M-registered-user tier (10% active in the peak hour ⇒ 100,000 active users, 30 actions each, 80% reads/20% writes ⇒ about 600,000 write actions per hour). The split between write actions is our planning estimate. The follow graph used in the experiments gives every user 40 followers, so one post produces 41 home-feed copies.

Step 1 · List your busiest writes

#	User action	SQL it runs (INSERT / UPDATE / DELETE, which tables)	Busy hour
1	Publish a post	`INSERT posts` (+ GIN search index), `INSERT post_tags` (one per hashtag), `INSERT feed_items` for the author and each of 40 followers (41 rows)	60,000 posts (≈2.8M rows)
2	Like a post	`INSERT likes` (one row per like; no counter `UPDATE`)	360,000
3	Log in	`INSERT sessions` (token hash, expiry)	100,000
4	Follow a user	`INSERT following` (follower side) + `INSERT followers` (followee side)	30,000 actions (60,000 rows)
5	Edit own post	`UPDATE posts` (body, `updated_at`; re-indexes the tsvector)	12,000

Hint applied (where writes come from): publishing is one click but 46 rows: by rows written it is the busiest action even though likes are more frequent. Registration (`INSERT user_directory` + `INSERT users`) is rarer (≈2,000/h) and is the sixth write.

Step 2 · Find the owner of each write, and pick the shard key

#	Rows that must change together	Owner	Owner's id column
1	the post, its `post_tags`, the author's own feed copy	the author (user)	`posts.author_id`, `post_tags.author_id`
	each follower's feed copy (written second, retried)	that follower	`feed_items.owner_id`
2	the like row with its post (counted with the post)	the post's author	`likes.author_id` (new column)
3	the session row	the user logging in	`sessions.user_id`
4	`following` row; `followers` row (two owners, two writes)	follower; followee	`following.follower_id`; `followers.followee_id`
5	the post row	the author	`posts.author_id`

Our shard key: the owning user's id (`users.id`, a UUID; for content rows the *author's* id), hashed into 64 fixed buckets with `shard_bucket()`.

Test the key: ✓ Every busy write already knows this value (the user is in the session; a like carries the post's `author_id`, which the app displayed with the post). ✓ The value never changes (a random UUID, not a name, email or category). ✓ It has many values that are not time ranges (400 test users, 1M planned; hashed, not sign-up order).

Hint applied (keep together what is written together): every child table carries the owner's id: `post_tags.author_id`, `likes.author_id`, `feed_items.owner_id`, `sessions.user_id`.

Step 3 · Check for a hot owner

- Busiest single owner: a campus celebrity account (athletics or a student-government account) with most of campus following it, during a game-day post.
- What it writes: one post fans out to every follower's feed (up to 10^5 copies); then thousands of likes per minute on that one post, all on the author's shard.
- Row locked by every one of those writes: none, by design: likes are rows, not `UPDATE posts SET like_count = like_count + 1`; the fan-out inserts distinct feed rows.
- Our plan: ☒ insert rows instead of updating one counter; ☒ accept the fan-out cost at class scale and say why: our test graph has a uniform 40 followers per user; for a real celebrity we would switch that author to *collect at read time* (lecture slide 50) instead of copying to 10^5 feeds.

Step 4 · Find the pages that read across owners

Page	Rows it needs live on…	Fix
Home feed	posts of everyone I follow (every shard)	☒ copy a small row (`feed_items`: id, author, 80-char preview) to the reader's shard, written second, retried
"Who I follow" / "my followers"	my follow edges	☒ copy: `following` on my shard, `followers` on theirs
Post search (M5)	posts on every shard	☒ ask every shard and combine (or a separate search index)

Shared, read-mostly tables copied to every shard: none needed (Campus Pulse has no product catalogue; the bucket function is the only thing identical everywhere).
Values unique across the whole app: **email** → `m6.user_directory` on shard_0.

Step 5 · Write the placement plan

Table	Lives on	Routed by	New column needed?
`users`	owner's shard	`id`	id becomes `uuid`; shard check
`sessions`	owner's shard	`user_id`	token carries the shard name
`posts`	author's shard	`author_id`	`uuid` id; `unique(author_id,id)`
`post_tags`	author's shard	`author_id`	yes: `author_id`
`likes`	post author's shard	`author_id`	yes: `author_id`
`following`	follower's shard	`follower_id`	replaces `friendships`
`followers`	followee's shard	`followee_id`	new table (other side)
`feed_items`	reader's shard	`owner_id`	new table (copy)
`user_directory`	directory (shard_0)	`email`	new table
`bucket_map`	directory (shard_0)	`bucket`	new table (64 rows)

Buckets: 64 (never changes). Shards: 2 → shard_0 owns buckets 0–31, shard_1 owns 32–63.

Before Part 2: ✓ every id column is a `uuid`; ✓ every owned table has the owner's id; ✓ the shard check is on `users`; ✓ the busiest transaction (publish) touches one shard for the real record (the follower copies are written second); ☐ instructor/TA sign-off: _____

2 Data models before and after

2.1 Before (1): the M1/M2 baseline schema

The deployed M1 application (`app/models.py`, schema `public`) stores everything in one Supabase PostgreSQL database with auto-increment integer ids:

Table	Columns	Notes
`users`	`id serial PK`, `email unique`, `display_name`, `password_hash`, `created_at`	
`sessions`	`id serial PK`, `user_id FK`, `token_hash unique`, `expires_at`	
`friendships`	`id serial PK`, `user_id FK`, `friend_id FK`, unique pair	one direction
`posts`	`id serial PK`, `author_id FK`, `body`, `created_at`, `updated_at`	
`likes`	`id serial PK`, `user_id FK`, `post_id FK`, unique pair	

Every write in this model is one or two small rows, and M2 showed that the hosted write path (`POST /api/messages`) saturated the single Render API process at about 15 requests/s long before the database was busy. To study write scaling we therefore had to *extend the database so that writes create the bottleneck* (Section 2.2).

2.2 Before (2): the extended single-database model used as the M6 baseline

We extended the schema with three features a campus feed needs, each of which multiplies the work behind one click (lecture slide 7: lock, change the table, update every index, log and wait for disk):

1. **Home feed by copy at write time** (`feed_items`): publishing a post inserts one feed row for the author and one for each follower (40 in our graph), so a post writes 41 rows into a table whose primary key is `(owner_id, created_at, post_id)`.
2. **Real-time full-text search**: `posts.search` is a generated `tsvector` with a GIN (inverted) index, maintained on every insert/edit; this replaces M5's 5-minute materialized-view refresh.
3. **Hashtags** (`post_tags`) parsed from the body, and **follows** as two tables (`following`/`followers`) instead of `friendships`.

We also applied the sharding prerequisites (UUID ids, owner id in every child table, one stored procedure per busy write) *already in the baseline*, so that the before/after comparison changes exactly one thing: the number of databases. In the baseline, `bucket_map` sends all 64 buckets to shard_0 and its check allows buckets 0–63.

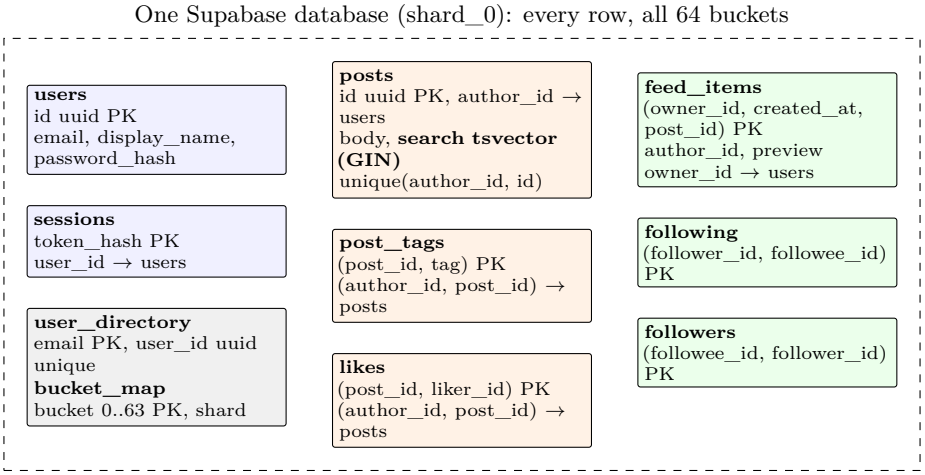


Figure 1: Before: the extended single-database model (schema `m6`). Blue = owner tables; orange = rows owned by the author; green = copies and follow edges; grey = directory.

2.3 After: the same tables, hash-partitioned over two databases

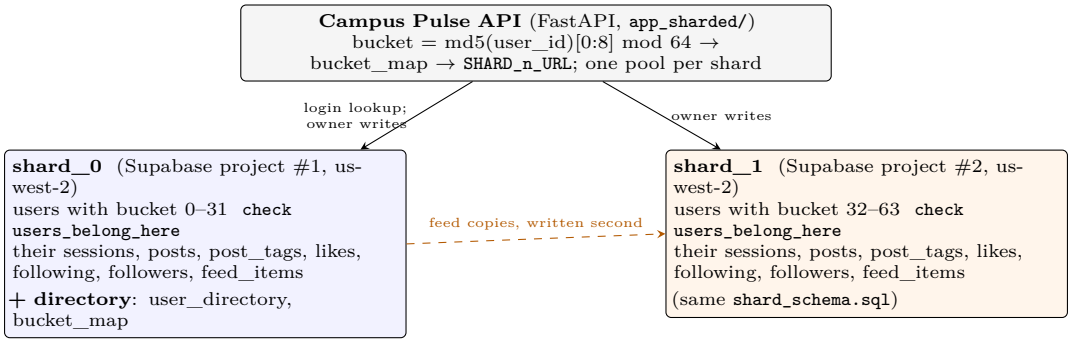


Figure 2: After: two Supabase projects in the same region. Each database owns half of the users and every row those users own. A post commits on the author's shard; copies for followers on the other shard are written second (and retried) by `m6.fanout()`.

Differences between the before and after models, table by table:

- **Same shape, different rows:** all eight data tables exist on both shards; a row's home is computed from its owner's id. Integer serial ids (M1) became UUIDs so no shared counter is needed.

- **New routing columns:** `likes.author_id` and `post_tags.author_id` let child rows follow the post's owner; composite foreign keys `(author_id, post_id)` → `posts(author_id, id)` guarantee a child can only attach to a post on the same shard.
- **Cross-owner relations became two rows:** M1's `friendships` became `following` (follower's shard) + `followers` (followee's shard); `likes.liker_id` has no foreign key because the liker may live elsewhere.
- **New central-but-small tables** on shard_0 only: `user_directory` (email uniqueness and login lookup) and `bucket_map`. They are touched only at registration and login.
- **Fail loudly:** each shard's `users` table rejects ids from the other shard's buckets.

3 Implementation: the ten steps on Supabase

We followed "How to shard in Azure + Supabase" step by step and adapted the handout `sharding_setup.sql` (Part A and the social-media appendix). Our app runs on Render, not Azure App Service, and for the load tests it ran on a lab server, so "Azure app settings" became environment variables (Step 7). All objects live in a dedicated schema `m6` so the M1/M2 tables in `public` keep serving the hosted M1 application.

Step	Lecture instruction	What we did (file)
1	Confirm writes are the bottleneck	`pg_stat_statements` (already enabled in schema `extensions`), reset before each series; the handout query plus a top-statement query (`m6/sql/step1_measure.sql`); results in Section 5.2.
2	Write down where every table goes	Worksheet Step 5 (Section 1); 64 buckets.
3	One Supabase project per shard, same region	`campus-pulse-csce553` (shard_0) and `campus-pulse-shard-1` (shard_1), both **West US (Oregon), us-west-2**, Nano compute, free plan.
4	Same schema file on every shard	`m6/sql/shard_schema.sql` (bucket function, 8 tables, `publish_post()`, `fanout()`).
5	Check that rejects other shards' users	`m6/sql/shard_check.sql`: `users_belong_here` with 0–31 / 32–63 (0–63 in the one-database baseline).
6	Directory and bucket map on shard_0	`m6/sql/directory.sql`: `user_directory`, `bucket_map` (32/32).
7	Connection strings in settings, not code	`SHARD_0_URL`, `SHARD_1_URL` in `.env` (git-ignored); the app maps the `bucket_map` name `shard_1` to `SHARD_1_URL`. Session-pooler strings (IPv4); the app uses the same pooler's transaction port 6543.
8	Load shared products on every shard	Not applicable: Campus Pulse has no shared catalogue. The only thing that must be identical everywhere is `shard_bucket()`, which we verify (Python = SQL for 2,000 ids on every shard).
9	Writes: route by owner	register (directory row, then user row on its shard, retried with the same id); publish (`publish_post()` on the author's shard, then `fanout()` copies on other shards); like (row on the post author's shard); follow (two idempotent writes). `app_sharded/main.py`
10	Reads: find the shard, then plain SQL	login looks up `user_directory`⋈`bucket_map` once; the session token is `shard_n.<random>`, so every per-user page (home feed, following, my posts) is a one-shard query.

Adapted SQL (excerpts)

The bucket rule and the shard check are unchanged from the handout except for the schema name:

```
create or replace function m6.shard_bucket(u uuid) returns int language sql immutable as $$
  select (('x' || substr(md5(u::text), 1, 8))::bit(32)::bigint % 64)::int $$;
alter table m6.users add constraint users_belong_here
  check (m6.shard_bucket(id) between 0 and 31);          -- 32..63 on shard_1
```

Publishing is the social-media equivalent of the handout's `checkout()`: one stored procedure, one local transaction on the owner's shard. It returns the followers that live on other shards so the app can write their copies second:

```
create or replace function m6.publish_post(p_post uuid, p_author uuid, p_body text, p_tags text[],
    out post_created_at timestamptz, out remote_followers uuid[]) language plpgsql as $$
```

```
begin
  post_created_at := clock_timestamp();
  insert into m6.posts (id, author_id, body, created_at, updated_at)
  values (p_post, p_author, p_body, post_created_at, post_created_at);
  insert into m6.post_tags (author_id, post_id, tag)
  select p_author, p_post, t from unnest(p_tags) t on conflict do nothing;
  insert into m6.feed_items (owner_id, created_at, post_id, author_id, preview)      -- local copies
  select x.owner, post_created_at, p_post, p_author, left(p_body, 80)
  from (select p_author as owner union
        select f.follower_id from m6.followers f where f.followee_id = p_author) x
  where exists (select 1 from m6.users u where u.id = x.owner);
  select coalesce(array_agg(f.follower_id), '{}') into remote_followers       -- for fanout()
  from m6.followers f where f.followee_id = p_author
   and not exists (select 1 from m6.users u where u.id = f.follower_id);
end $$;
```

Registration follows Step 9 exactly (A on the directory, B on the returned shard), and a like is the appendix statement with the routing column added:

```
insert into m6.user_directory (email) values (lower(:email)) returning user_id;   -- A, shard_0
insert into m6.users (id, email, display_name, password_hash) values (...)        -- B, user's shard
  on conflict (id) do nothing;                                                    -- retry-safe
insert into m6.likes (author_id, post_id, liker_id) values (...) on conflict do nothing;
```

4 Load-test method

System under test. The load driver and the API run on one lab server (32 cores, 188 GB RAM). The API is `app_sharded` under uvicorn with 6 worker processes. Each worker holds a `psycopg` pool of up to 32 connections per shard to the Supabase transaction pooler (port 6543), so a project sees at most 192 client connections, below the pooler's limit of 200; Supabase maps them onto 15 server connections per Nano project. Databases: Supabase free plan, Nano compute (2 vCPU), PostgreSQL 17, both in us-west-2 (Render's default Oregon region is the same). The median `SELECT 1` round trip from the server was 73 ms to shard_0 and 75 ms to shard_1. We sized the API so that it is not the first limit and checked this (Section 5.1).

Two series, one variable. Series **S1 (before)**: `ACTIVE_SHARDS=1`, all 64 buckets on shard_0. Series **S2 (after)**: `ACTIVE_SHARDS=2`, buckets 0–31 on shard_0 and 32–63 on shard_1. Both series use the same code and the same seed: each rebuilds schema `m6` and loads the same 400 users (fixed random seed 553), the same 16,000 follow edges (40 per user) and one session token per user, then runs `m6/run_series.py`. SHA-256 hashes of the code are stored with the results.

Driver. `m6/load_m6.py` keeps the M2 driver's closed-loop design and statistics: warm-up discarded, fixed measurement window, per-request CSV, p50/p95/p99/max and error rate. Sharding needed two changes. Every client is a *different* user, because one shared token, as in M2, would put all writes on one owner and therefore on one shard. Concurrency goes up to 256.

Stages. The publish staircase uses $c = 1, 4, 8, 16, 32, 64, 128, 256$ clients. Each stage has 5 s of warm-up and 30 s of measurement; then all posts and feed rows are truncated and the system recovers for 15 s, so every stage starts with the same table sizes. The stop rule is the same as in M2: error rate >10% or p99 >5 s (never triggered). After the staircase come a repeat of $c = 128$, and like-only, home-feed-read and a 70/20/10 read/like/post mix at $c = 128$, run against 2,000 pre-loaded like targets and pre-filled feeds. We later added intermediate publish stages to locate each peak, with the same protocol (`m6/run_extra.py`): $c = 48, 96$ for S1, and $c = 96, 160, 192$ plus a repeat of $c = 256$ for S2. The 30 s window (M2 used 60 s) keeps one-database stages under the free plan's 500 MB limit: each post leaves ≈13 KB of rows and index entries, and the single database reaches ≈575 posts/s.

Database-side evidence. During every measured window, on every shard, the driver records:

- 1 Hz samples of `pg_stat_activity`: active client backends and their wait events;
- before/after deltas of three statistics views: commits and rows inserted (`pg_stat_database`), WAL bytes (`pg_stat_wal`), and the calls and execution time of every `m6` statement (`pg_stat_statements`).

Host CPU, iowait and disk writes of each database VM come from Supabase's Prometheus endpoint (`/customer/v1/privileged/metrics`), which refreshes about every 60 s and is logged continuously by `m6/metrics_logger.py`. API CPU is the sum over the uvicorn master and its workers (psutil, sampled every 2 s).

Functional check. `m6/smoke_test.py` exercises Steps 9–10 over HTTP on the two-shard layout: registration lands users on both shards, a duplicate email is rejected, login returns the user's shard, a cross-shard follow writes both rows, a post's feed copy is written on the other shard, likes go to the author's shard, only the author can edit, forged tokens are rejected, and a wrong-shard insert is rejected. All 16 checks pass (`results/smoke-S2.json`).

Other controls. M5's pg_cron job (refresh `search.post_search` every 5 minutes) was paused during each series and re-enabled afterwards. The M1/M2 tables in `public` and the hosted Render service were not modified.

5 Results

5.1 First, make sure the database is what saturates

Our first two attempts did not measure the database. With 8 API workers × 8 pooled connections the publish curve flattened at ≈300 posts/s, but the database had only 3.5–5 busy backends and a home-feed *read*, which barely touches the database, was capped at the same ≈380 requests/s: the API's connection pool was the bottleneck (each request holds a connection for a 75 ms round trip). Raising the pool to 16×12 moved the plateau to ≈460 posts/s with the same symptom. With the final setting (6 workers × 32 connections per shard; at most 192 client connections per project, below the Supabase pooler's 200) a feed read reaches 1,571 requests/s and a like 1,330 requests/s through the same API with the database almost idle, while publish saturates at ≈574 posts/s. The request path can therefore carry at least 2.7× the publish throughput we report, so the publish plateau below is a database limit. The superseded runs are kept in `m6/results-run1-apipool/` and `m6/results-run2-api16x12/`.

5.2 Before sharding: writes are the bottleneck

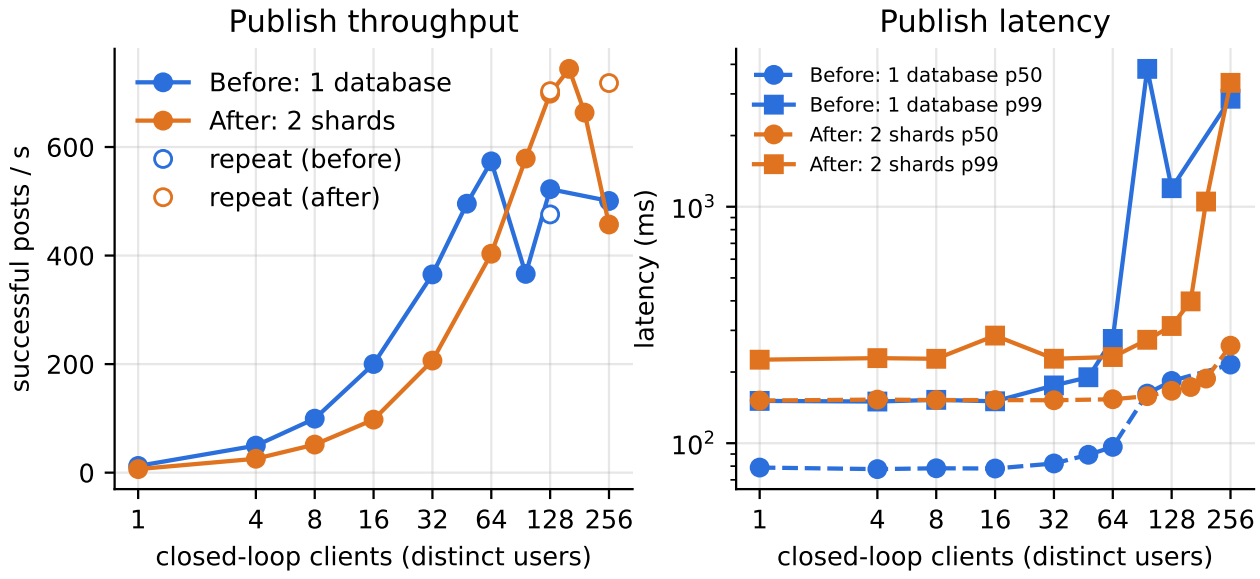


Figure 3: Publish staircase, one database (blue) vs. two shards (orange). Hollow points are repeated stages. Each point is a 30 s closed-loop stage with distinct users; zero errors in every stage.

Throughput and latency. With one database, publish throughput grows linearly up to 32 clients (365 posts/s, p99 176 ms), peaks at **574 posts/s at 64 clients** (25,600 rows inserted/s, 10.8 MB/s of WAL), and then *falls* as more writers are added: 522 posts/s at 128 clients and 501 at 256, while p99 rises from 276 ms to 1.2 s and 2.9 s. The 96-client stage was unstable (366 posts/s, p99 3.8 s). The repeated 128-client stage gave 476 posts/s, 9% below the first run; the shared Nano instance is noisy once it is overloaded. More clients past the knee only add queueing; Little's law holds in every stage (Table 1).

Where the time goes inside the database. The 1 Hz `pg_stat_activity` samples (Figure 4) show the number of busy backends rising from 1.5 (16 clients) to 14–15.6 (64 clients and above), which is the Supabase pooler's 15 server connections for a Nano project. On a 2-vCPU machine, 7–10 of them are runnable on CPU, and 3–4 wait on `LWLock:WALWrite` / `IO:WalSync`, i.e. for their commit record to reach disk, plus 1–3 on buffer-content and relation-extension locks. The same statement got slower as load rose: the mean execution time of `publish_post()` went from 3.4 ms (16 clients) to 7.0, 11.6, 16.8 and 22–24 ms (32, 48, 64, ≥128 clients). The work per post did not change (46 rows, one commit), so this 7× slowdown is contention.

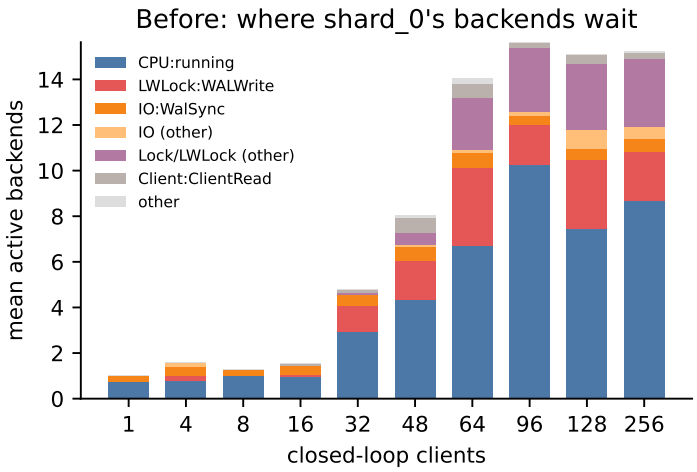


Figure 4: One database: mean number of active backends by wait event (1 Hz samples). Blue = on CPU; red/orange = waiting for the WAL to be written and flushed (commit).

The host. Supabase's metrics endpoint refreshes about every 60 s, so we logged it continuously and plot it against the stage windows (Figure 5). During the saturated publish stages (64–256 clients) the database VM spent 34–61% of its CPU in user+system time and another 18–44% in **iowait**; busy+iowait stayed at 73–79%, leaving about 20% idle, and it wrote 16–21 MB/s to disk. The two cores are either computing inserts and index updates or waiting for the disk.

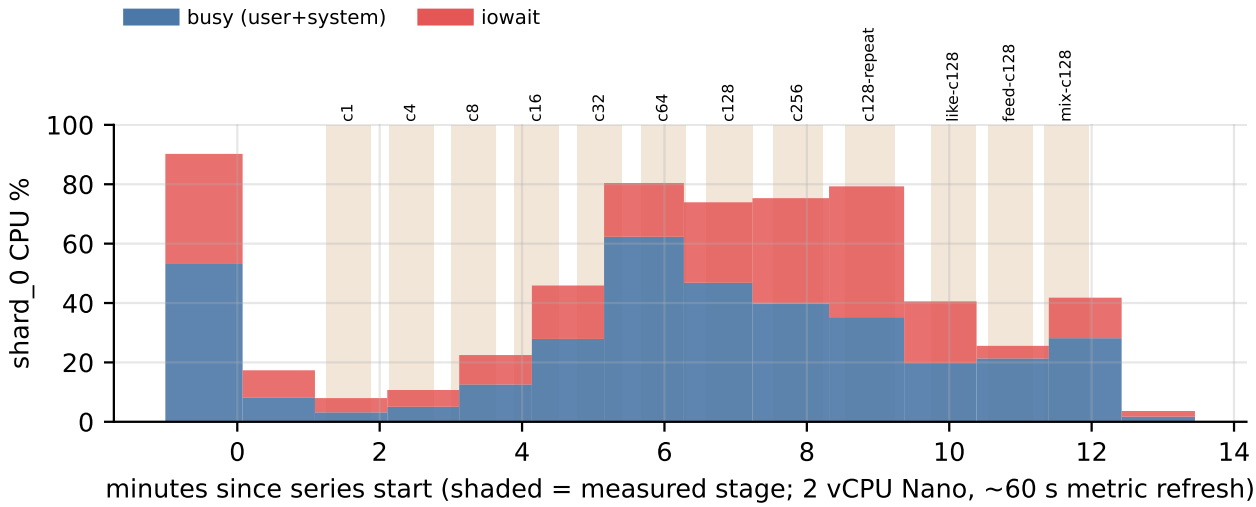


Figure 5: One database: shard_0 VM CPU per metrics refresh (blue = user+system, red = iowait). Shaded bands are the measured stages.

Step 1 query. The handout query (statements starting with INSERT/UPDATE/DELETE) returns no rows for Campus Pulse, because every busy write runs inside a stored procedure called with `SELECT`. Over all statements, `publish_post()` accounts for 98.8% of the execution time of the top ten statements and wrote 1.53 GB of WAL during the one-database staircase:

Shard	Statement (`pg_stat_statements`, after the S1 staircase)	Calls	Total ms	Mean ms	WAL MB
shard_0	`select post_created_at, remote_followers from m6.publish_post(`	81,792	1,261,903	15.43	1,529.1
shard_0	`select coalesce(wait_event_type,$1), coalesce(wait_event,$2),`	251	4,720	18.80	0.0
shard_0	`truncate m6.feed_items, m6.likes, m6.post_tags, m6.posts`	8	3,462	432.70	0.2
shard_0	`SELECT * FROM pgbouncer.get_auth($1)`	55	2,805	51.00	0.0
shard_0	`select u.id, u.display_name from m6.sessions s join m6.users u`	4,556	2,604	0.57	0.0
shard_0	`DISCARD ALL`	33	776	23.51	0.0

Conclusion for Step 1: writes are the bottleneck. The top statement is the write path, the database runs out of connections, CPU and commit bandwidth, more clients make throughput worse, and the cheaper fixes from slide 25 are already in place (one stored-procedure call per post, no counter row, no unused index). The next cheapest fix would be a bigger compute plan; the assignment asks us to shard.

5.3 After sharding: the same staircase on two databases

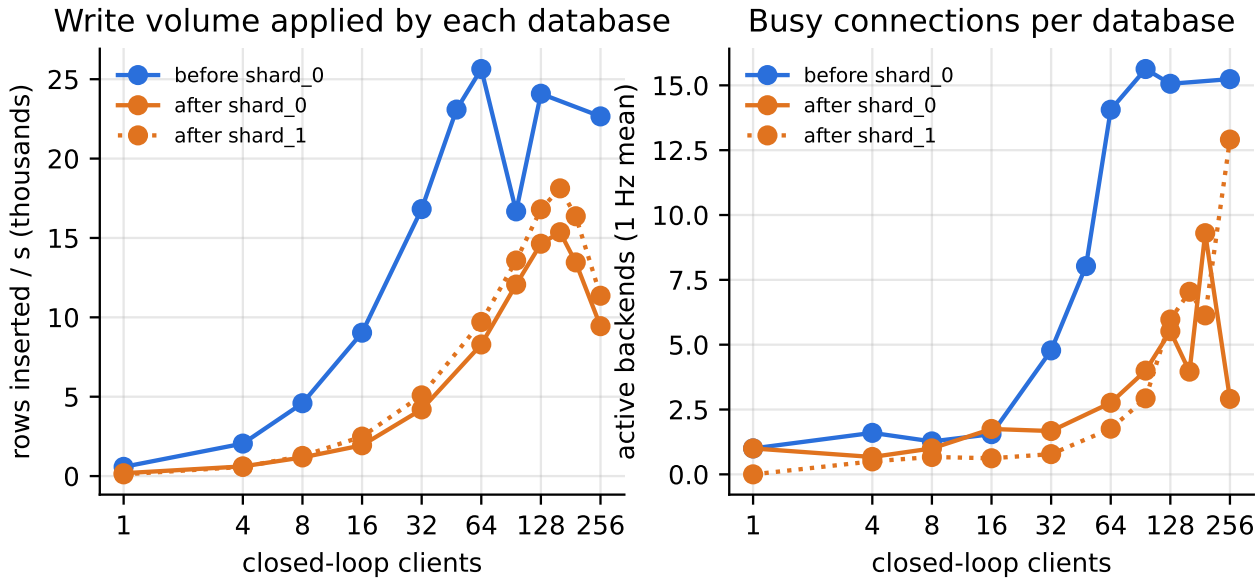


Figure 6: Rows inserted per second and busy backends, per database. After sharding, each database applies about half of the rows, and neither database reaches the 15-backend ceiling at the throughputs where the single database is saturated.

Capacity. With two shards, publish peaks at **744 posts/s at 160 clients** (p99 398 ms), and stays at 698–744 posts/s from 128 to 160 clients with p99 $\leq$ 400 ms. The one-database peak was 574 posts/s with p99 276 ms. The peak is 1.30× higher, and over the saturated region the median is 700 vs. 501 posts/s (1.40×). At 192 and 256 clients the two-shard system also overloads (663 and 457/718 posts/s, p99 up to 3.3 s; the two 256-client runs differ by 57%).

Load per database. Compare stages with the same throughput: one database at 64 clients (574 posts/s) and two shards at 96 clients (579 posts/s). Each shard inserted 12.1k and 13.6k rows/s instead of 25.6k, had 4.0 and 2.9 busy backends instead of 14.1, and ran `publish_post()` in 3.5 ms instead of 16.8 ms. Halving the rows per database removed most of the contention, so each shard is far below its own saturation point while carrying the same user load.

Latency at low load got worse. At 1–64 clients, two shards deliver about half the throughput of one database at the same concurrency (403 vs. 574 posts/s at 64 clients), and p50 rises from 78 to 153 ms. The cause is the cross-shard copy: a post now commits on the author's shard and then writes follower copies on the other shard, written second as the lecture requires, so one click costs two sequential 75 ms round trips. Closed-loop throughput at fixed concurrency is c/R, so it halves until the databases saturate.

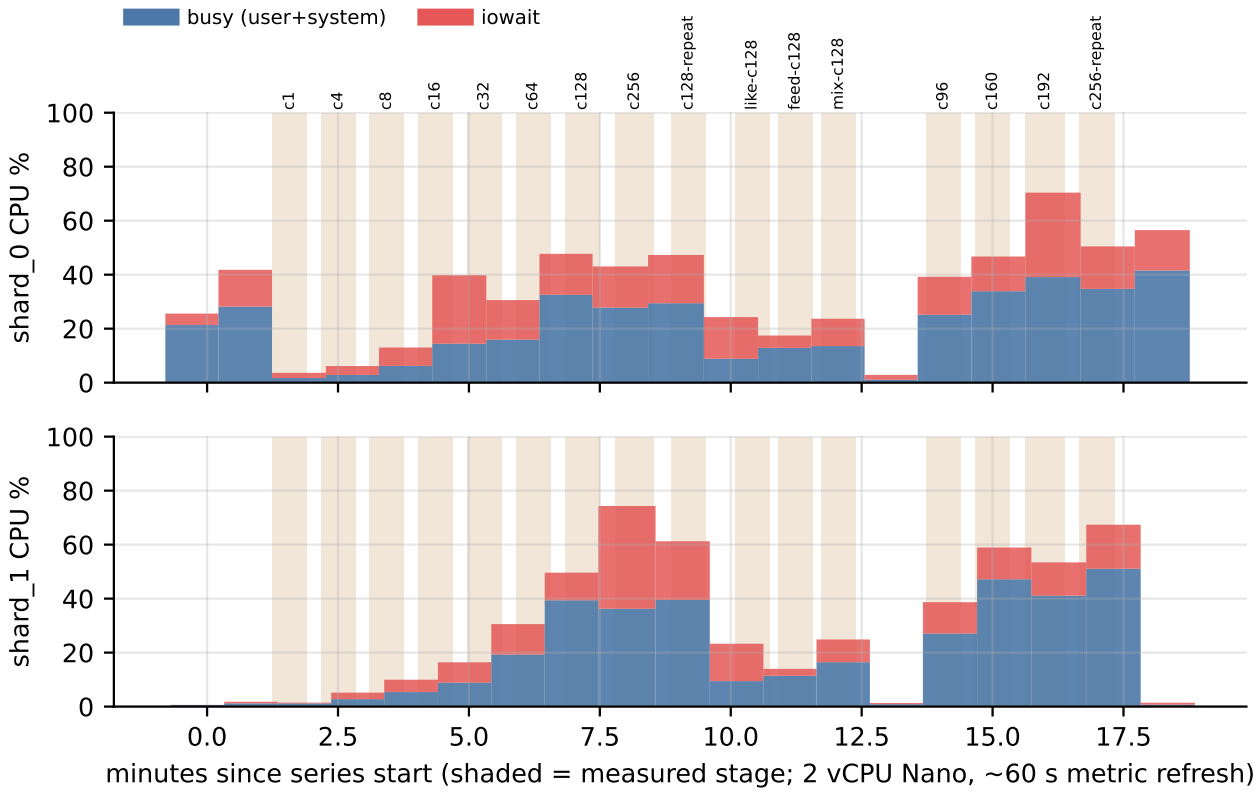


Figure 7: Two shards: VM CPU of both databases. In the 128-client stages both are around 45–50% busy+iowait; the first 256-client stage drove shard_1 to 74% (iowait 38%) while shard_0 stayed near 43%.

The busiest shard decides. shard_1 owns 219 of the 400 users (55%; buckets are hashed, so the split is close but not exact) and therefore receives 24 rows per post vs. 21 on shard_0. In the first 256-client stage shard_1's `publish_post()` took 22.9 ms and it had 12.9 busy backends, while shard_0 had 2.9. Every post waits for both shards, so the slower shard set the throughput (457 posts/s). This is lecture slide 15: capacity is the busiest shard.

Other workloads (128 clients).

Workload	1 DB req/s	2 shards req/s	Change	p99 (1 DB → 2)	Note
Like (1-row insert)	1,330	1,609	+21%	188 → 148 ms	commit waits gone; API ceiling
Home-feed read	1,571	1,636	+4%	149 → 148 ms	DB nearly idle in both
70/20/10 mix	1,553	1,484	−4%	150 → 160 ms	posts pay a 2nd round trip

Reads did not get faster, and the lecture says they should not: sharding adds write capacity. The home feed remained a one-shard query because its rows are copies on the reader's shard.

5.4 Correctness checks (lecture slide 41)

Check	Result (two shards)	Expected
Users per shard (`count(*)` on each)	shard_0: 181, shard_1: 219	close, not identical
Users per shard via directory ⋈ bucket_map	shard_0: 181, shard_1: 219	same as above
Rows on the wrong shard	0 and 0	0
`bucket_map`	32 / 32 buckets	32 / 32
Bad insert (bucket 37) on shard_0	rejected: violates `users_belong_here`	rejected
Python `shard_bucket` vs. SQL, 2,000 ids	identical on both shards	identical

5.5 All stages

Per-stage results; for two shards, per-database columns are given as shard_0 / shard_1. "Exec" = seconds of statement execution per second (from `pg_stat_statements`); "Busy" = mean active backends; API CPU in % of one core.

Workload	c	Requests	OK/s	p50	p95	p99	max	Err	Rows/s (k)	Exec s/s	Busy	WAL MB/s	API
Before: one database (S1)													
post	1	372	12.4	78.9	84.1	151.0	158	0.00%	0.6	0.04	1.0	0.2	3
post	4	1,497	49.8	77.7	83.4	150.0	224	0.00%	2.0	0.14	1.6	0.8	6
post	8	2,992	99.5	78.3	83.8	152.5	249	0.00%	4.6	0.34	1.3	1.9	11
post	16	6,021	200.2	78.2	83.4	150.4	333	0.00%	9.0	0.69	1.6	3.7	20
post	32	11,001	365.2	82.1	101.3	175.5	954	0.00%	16.8	2.59	4.8	6.9	37
post	48	15,023	495.6	89.3	146.3	190.1	1,489	0.00%	23.1	5.74	8.0	9.7	50
post	64	17,277	573.5	96.5	169.7	276.0	1,386	0.00%	25.6	9.67	14.1	10.8	55
post	96	11,327	366.2	162.1	303.7	3,827.8	10,794	0.00%	16.7	13.11	15.6	7.0	37
post	128	16,376	522.3	183.4	502.9	1,197.8	3,344	0.00%	24.1	11.74	15.1	10.1	52
post	256	16,211	500.6	215.1	1,350.8	2,861.8	6,903	0.00%	22.7	11.71	15.2	9.6	59
post (repeat)	128	14,710	475.8	192.8	712.6	1,224.9	2,796	0.00%	21.7	11.45	15.8	9.1	47
like	128	40,072	1,329.6	76.4	158.8	188.5	354	0.00%	1.1	0.10	2.7	0.4	122
feed	128	47,306	1,571.4	75.8	109.8	148.5	238	0.00%	0.0	0.23	3.3	0.0	240
mix	128	46,818	1,552.9	76.2	109.9	149.6	247	0.00%	7.2	0.93	2.9	2.9	212
After: two shards (S2)													
post	1	196	6.5	151.5	156.0	225.5	240	0.00%	0.2 / 0.1	0.01 / 0.00	1.0 / 0.0	0.1 / 0.0	2
post	4	773	25.6	152.9	164.4	229.0	252	0.00%	0.6 / 0.6	0.04 / 0.04	0.7 / 0.5	0.3 / 0.2	5
post	8	1,557	51.7	152.2	157.3	227.5	232	0.00%	1.2 / 1.3	0.07 / 0.06	1.0 / 0.7	0.5 / 0.5	9
post	16	2,975	97.7	152.2	189.7	285.2	1,948	0.00%	1.9 / 2.5	0.53 / 0.14	1.8 / 0.6	0.8 / 1.0	14
post	32	6,226	206.5	151.9	158.4	227.6	441	0.00%	4.2 / 5.1	0.30 / 0.26	1.7 / 0.8	1.8 / 2.2	27
post	64	12,163	403.4	153.4	178.1	231.4	378	0.00%	8.3 / 9.7	0.79 / 0.59	2.8 / 1.8	3.6 / 4.2	52
post	96	17,451	578.8	158.1	210.7	273.7	718	0.00%	12.1 / 13.6	1.54 / 1.54	4.0 / 2.9	5.3 / 6.0	75
post	128	21,205	698.3	166.6	257.5	313.1	926	0.00%	14.6 / 16.8	2.59 / 2.25	5.5 / 6.0	6.4 / 7.4	90
post	160	22,532	744.0	172.8	352.5	397.9	1,909	0.00%	15.4 / 18.1	2.39 / 3.67	4.0 / 7.0	6.7 / 8.0	92
post	192	20,188	663.4	187.7	546.3	1,051.7	2,309	0.00%	13.5 / 16.4	5.79 / 3.52	9.3 / 6.1	6.0 / 7.7	82
post	256	14,954	457.2	258.5	1,727.3	3,341.3	4,955	0.00%	9.4 / 11.4	1.16 / 8.77	2.9 / 12.9	4.2 / 5.0	59
post (repeat)	128	21,222	702.9	166.5	260.6	309.6	750	0.00%	14.2 / 17.6	2.13 / 3.11	3.8 / 6.3	6.3 / 7.8	87
like	128	48,390	1,608.7	75.5	99.8	147.9	185	0.00%	0.6 / 0.7	0.05 / 0.07	0.9 / 1.5	0.2 / 0.3	130
feed	128	49,210	1,636.2	75.5	91.1	148.1	167	0.00%	0.0 / 0.0	0.11 / 0.10	0.4 / 0.6	0.0 / 0.0	244
mix	128	44,730	1,483.5	75.7	153.6	159.7	263	0.00%	3.2 / 3.8	0.31 / 0.37	0.8 / 2.3	1.4 / 1.6	204
post (repeat)	256	21,998	717.9	181.8	798.5	916.4	1,471	0.00%	14.6 / 18.0	2.56 / 4.81	5.0 / 7.9	6.4 / 7.9	88

Table 1: Little's law check: in-flight requests $L = X \cdot R$ match the closed-loop client count c.

Shards	Workload	c	X (req/s)	R mean (ms)	$L = XR$
1	post	16	200.2	79.8	15.97
1	post	64	573.5	111.3	63.83
1	post	256	500.6	496.1	248.36
2	post	16	97.7	162.0	15.83
2	post	64	403.4	158.2	63.80
2	post	256	457.2	542.0	247.83

6 Discussion

Why not 2×? Sharding split the rows, not the commits. Per post, the one database applied 46 rows in 1.02 commits. After sharding each database applied 21 or 24 rows per post, about half, but still 1.03 commits per post: the author's shard commits `publish_post()`, and the other shard commits `fanout()` for the followers who live there. With 40 random followers per user, almost every post has followers on both shards. Commit-related waits (WALWrite, WalSync) were a large share of the single database's saturation (Figure 4), and the commit rate each Nano disk must sustain went from 587/s (one database at its peak) to 765/s per shard (two shards at their peak). Sharding therefore removed most of the CPU and lock contention from inserting and indexing rows. It did not remove the per-commit cost. The measured gain is 1.30–1.40×, not the ideal 2× of an app whose writes stay with one owner (lecture slide 9, e-commerce).

The cost of copy-at-write across shards. The same design decision has a latency cost: one publish is two sequential round trips (Section 5), so at low and medium load users see a p50 of about 155 ms instead of 78 ms. The lecture accepts this ordering because the real record has to exist before its copies. Two fixes follow from the lecture's own principles:

1. *Combine many small writes into fewer* (slide 25): queue the follower copies and apply them per shard in batches, e.g. one `fanout()` transaction per shard every 50 ms carrying many posts. The post returns after one round trip, and the second shard's commit rate drops from one per post to one per batch.
2. *Collect at read time* for authors with many followers (slide 50): no copies at all, so the publish is a one-shard, one-commit transaction, and the feed read asks both shards and merges.

We did not implement these. They change the semantics (feeds become eventually consistent within the batch interval), and the assignment asks for the lecture's ten steps.

Hot shard and hot owner. With 400 users, hashing gave a 45/55 split, and the busier shard limited the overloaded 256-client stage. More users would even this out; the lecture's 2,000-user test gave 1,008:992. One very busy owner, the worksheet's Step 3 celebrity, cannot be split by hashing: all of its likes land on one shard. Because likes are inserted as rows and never update a counter, they do not queue on a single row lock.

Revised capacity statement. With two Nano shards, Campus Pulse sustained about 700 posts/s (≈31,000–33,000 rows/s across both databases) with p99 $\leq$ 400 ms, against 574 posts/s with p99 276 ms on one Nano database. The M2 1M-user model needs about 17 posts/s (10% of 167 write RPS). Both configurations cover that, so for Campus Pulse at class scale sharding would be premature ("shard last", slide 25). The numbers show what sharding buys once a single database is saturated: about 1.3–1.4× for this fan-out-heavy write. Writes that touch one owner (likes, sessions) should scale closer to 2×, but our like test reached the API's ≈1,600 req/s ceiling before either shard saturated (+21%), so we did not measure their limit.

Confounds and limits.

- **Shared free-tier hardware.** Nano projects run on shared hosts. Overloaded stages varied by up to 57% between repeats (256 clients, two shards); unsaturated stages repeated within 1% (128 clients, two shards: 698 vs. 703 posts/s). We report repeats instead of averaging them.
- **30 s measurement windows** (M2 used 60 s) because a one-database stage writes ≈13 KB per post and must stay below the 500 MB free-plan limit. Every stage starts from truncated tables.
- **Host metrics have 60 s resolution and are not aligned with stages**, so we show them as a timeline and use the in-database 1 Hz samples for per-stage claims.
- **Two different Postgres patch versions** (17.6 on shard_0, 17.11 on shard_1); same region, same compute size; round-trip times 73 and 75 ms.
- **Load generator and API on one lab server**, 75 ms from Supabase. The API ceiling (about 1,600 req/s for requests that barely touch the database) is above every publish result we report. Hosted Render was not used for M6 because its single free instance tops out near 15 requests/s (M2).
- **Closed-loop, uniform workload**: 40 followers per user and one fixed post body with three hashtags. Real follower counts are skewed, which would make the hot-owner effects larger.

Housekeeping. The M4/M4b experiment schemas (about 415 MB) were exported to compressed CSV and dropped from shard_0 to make room; the M1/M2 `public` tables, the M5 `search` schema and the hosted app were not modified, and M5's cron refresh was re-enabled after each series.

7 Reproduction

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt -r m2/requirements.txt
# .env (git-ignored): SHARD_0_URL=postgresql://postgres.<ref0>:<pw>@aws-0-us-west-2.pooler.supabase.com:5432/postgres
#                     SHARD_1_URL=postgresql://postgres.<ref1>:<pw>@aws-0-us-west-2.pooler.supabase.com:5432/postgres
python m6/run_series.py --shards 1     # before: one database (rebuilds schema m6, seeds, runs all stages)
python m6/run_series.py --shards 2     # after: two shards
python m6/run_extra.py --shards 2 --steps 96,160,192 --repeat 256    # intermediate stages
```

```
python m6/run_extra.py --shards 1 --steps 48,96 --reseed
m6/serve.sh 2 2 8602 & python m6/smoke_test.py   # functional check of Steps 9-10 (2-shard layout)
python m6/report/build.py              # figures + tables from m6/results/*.json
cd m6/report && latexmk -pdf main.tex
# manual steps: python m6/shardctl.py setup|seed|check --shards 2 ; m6/serve.sh 2 6 8600
```

Raw evidence in `m6/results/`: per-request CSVs (`*.requests.csv`), 1 Hz wait-event samples (`*.dbwait.csv`), API CPU (`*.apicpu.csv`), per-stage JSON with database deltas, `summary.csv`, `timeline.jsonl` (UTC), the shard checks (`S1-check.json`, `S2-check.json`), the Step 1 statement statistics (`step1-S1.json`, `step1-S2.json`), the functional check (`smoke-S2.json`), the host-metrics log (`dbhost-timeline.jsonl`) and SHA-256 fingerprints of the code that produced them. Session tokens and connection strings stay in the git-ignored `m6/.private/` and `.env`.